

LIBRARY MANAGEMENT SYSTEM

MySQL Database Schema and SQL Project Report

Technology Stack

Frontend: HTML, CSS, JavaScript

Backend: Flask (Python)

Database: MySQL

Deployment Target: AWS EC2 + AWS RDS MySQL

Database Scope: 4 Core Tables

Users | Authors | Books | Borrowings

Table of Contents

1. Project Overview
2. Objectives and Scope
3. Technology Architecture
4. Database Design
5. Table Specifications
6. Complete MySQL Schema
7. Sample Data
8. CRUD Operations
9. Book Borrowing and Return Workflow
10. SQL Analysis and Reporting Queries
11. Flask Integration
12. AWS Deployment Design
13. Security and Data Integrity
14. Testing Checklist
15. Future Enhancements
16. Conclusion
17. Reference

1. Project Overview

The Library Management System is a full-stack web application designed to manage books, authors, library users, and borrowing transactions. The application will expose the MySQL database through a Flask backend and provide a browser-based interface using HTML, CSS, and JavaScript.

The database is intentionally limited to four core tables so that the project remains easy to understand, implement, and demonstrate during interviews. The design still supports normalized relationships, CRUD operations, borrowing and return workflows, availability tracking, and useful SQL analytics.

Design Reference

The project is inspired by the SQL-oriented library-management use case described by GeeksforGeeks, which demonstrates books, borrowers, issue/due/return dates and analysis such as top borrowers, most-borrowed books, late returns, and currently overdue books. This report adapts those ideas into a normalized four-table application schema suitable for a Flask web application.

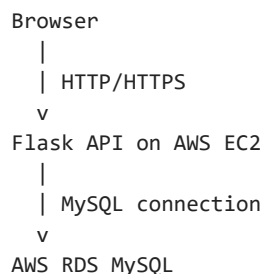
2. Objectives and Scope

- Maintain a catalog of books and authors.
- Register and manage library users.
- Track when books are issued and returned.
- Maintain the number of available copies of each book.
- Support create, read, update, and delete/deactivate operations.
- Generate useful reports such as most borrowed books and overdue books.
- Provide a database foundation that can be deployed on AWS RDS and accessed by Flask running on AWS EC2.

3. Technology Architecture

The application follows a simple three-layer architecture: presentation, backend/API, and database.

Layer	Technology	Responsibility
Frontend	HTML, CSS, JavaScript	Pages, forms, tables, search, dashboard and API calls
Backend	Python + Flask	REST endpoints, validation, business logic and database access
Database	MySQL	Persistent relational data, constraints, transactions and SQL reporting
Hosting	AWS EC2	Runs Flask application with Gunicorn and Nginx
Managed DB	AWS RDS for MySQL	Runs the production MySQL database



4. Database Design

4.1 Core Tables

Table	Purpose	Primary Key
users	Stores registered library members	user_id
authors	Stores book author information	author_id
books	Stores book catalog and copy availability	book_id
borrowings	Stores issue, due and return transactions	borrowing_id

4.2 Relationships

A user can create many borrowing records, while each borrowing record belongs to one user. An author can be associated with many books, while each book has one primary author in this simplified four-table version. A book can appear in many borrowing records over time.

USERS (1) -----< BORROWINGS >----- (1) BOOKS >----- (1) AUTHORS

users.user_id -> borrowings.user_id
 books.book_id -> borrowings.book_id
 books.author_id -> authors.author_id

4.3 Why these four tables?

Extra tables such as a separate category master, book-author junction table, or fines table were intentionally removed from the first version. Category is kept as a simple column in books, fines can be calculated from due_date and return_date, and the author relationship is simplified to one primary author per book. This keeps the application small while retaining meaningful SQL relationships.

5. Table Specifications

5.1 users

Column	Type	Constraint	Description
user_id	INT	PK, AUTO_INCREMENT	Unique user identifier
name	VARCHAR(100)	NOT NULL	Full name
email	VARCHAR(150)	NOT NULL, UNIQUE	Unique email address
phone	VARCHAR(15)	NULL	Contact number
status	ENUM	DEFAULT ACTIVE	ACTIVE or INACTIVE
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Registration timestamp

5.2 authors

Column	Type	Constraint	Description
author_id	INT	PK, AUTO_INCREMENT	Unique author identifier
author_name	VARCHAR(150)	NOT NULL	Author name

5.3 books

Column	Type	Constraint	Description
book_id	INT	PK, AUTO_INCREMENT	Unique book identifier
title	VARCHAR(200)	NOT NULL	Book title
isbn	VARCHAR(20)	UNIQUE	ISBN number

author_id	INT	FK, NOT NULL	References authors
category	VARCHAR(100)	NULL	Book category/genre
publisher	VARCHAR(150)	NULL	Publisher name
publication_year	YEAR	NULL	Year published
total_copies	INT	DEFAULT 1	Total physical copies
available_copies	INT	DEFAULT 1	Copies currently available

5.4 borrowings

Column	Type	Constraint	Description
borrowing_id	INT	PK, AUTO_INCREMENT	Unique transaction identifier
user_id	INT	FK, NOT NULL	References users
book_id	INT	FK, NOT NULL	References books
issue_date	DATE	DEFAULT CURRENT_DATE	Date issued
due_date	DATE	NOT NULL	Expected return date
return_date	DATE	NULL	Actual return date
status	ENUM	DEFAULT BORROWED	BORROWED or RETURNED

6. Complete MySQL Schema

The following script creates the database and all four tables. It also adds foreign keys, uniqueness rules, indexes and checks that protect data integrity.

```

CREATE DATABASE IF NOT EXISTS library_management;
USE library_management;

-- 1. USERS
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    phone VARCHAR(15),
    status ENUM('ACTIVE', 'INACTIVE') DEFAULT 'ACTIVE',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. AUTHORS
CREATE TABLE authors (
    author_id INT AUTO_INCREMENT PRIMARY KEY,
    author_name VARCHAR(150) NOT NULL
);

-- 3. BOOKS
CREATE TABLE books (
    book_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    isbn VARCHAR(20) UNIQUE,
    author_id INT NOT NULL,
    category VARCHAR(100),
    publisher VARCHAR(150),
    publication_year YEAR,
    total_copies INT NOT NULL DEFAULT 1,
    available_copies INT NOT NULL DEFAULT 1,

    CONSTRAINT fk_books_author
        FOREIGN KEY (author_id)
        REFERENCES authors(author_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT chk_total_copies
        CHECK (total_copies >= 0),

    CONSTRAINT chk_available_copies
        CHECK (available_copies >= 0),

    CONSTRAINT chk_available_not_exceed_total
        CHECK (available_copies <= total_copies)
);

-- 4. BORROWINGS
CREATE TABLE borrowings (
    borrowing_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    book_id INT NOT NULL,
    issue_date DATE NOT NULL DEFAULT (CURRENT_DATE),
    due_date DATE NOT NULL,
    return_date DATE NULL,
    status ENUM('BORROWED', 'RETURNED') DEFAULT 'BORROWED',

    CONSTRAINT fk_borrowings_user
        FOREIGN KEY (user_id)
        REFERENCES users(user_id)

```

```

        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT fk_borrowings_book
        FOREIGN KEY (book_id)
        REFERENCES books(book_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);

-- Indexes for common searches and joins
CREATE INDEX idx_books_title ON books(title);
CREATE INDEX idx_books_author ON books(author_id);
CREATE INDEX idx_borrowings_user ON borrowings(user_id);
CREATE INDEX idx_borrowings_book ON borrowings(book_id);
CREATE INDEX idx_borrowings_status ON borrowings(status);
CREATE INDEX idx_borrowings_due_date ON borrowings(due_date);

```

6.1 Important Integrity Rules

- email is unique so one account cannot be registered twice with the same email.
- ISBN is unique so the same book identifier cannot be duplicated.
- available_copies can never be negative or greater than total_copies.
- Foreign keys prevent borrowings from referring to users or books that do not exist.
- RESTRICT on delete protects borrowing history from accidental deletion.

7. Sample Data

7.1 Insert Authors

```

INSERT INTO authors (author_name) VALUES
('Robert C. Martin'),
('James Clear'),
('Martin Fowler'),
('Joshua Bloch'),
('Andrew Hunt');

```

7.2 Insert Users

```

INSERT INTO users (name, email, phone, status) VALUES
('Dharshan J', 'dharshan@example.com', '9876543210', 'ACTIVE'),
('Arun Kumar', 'arun@example.com', '9876543211', 'ACTIVE'),
('Priya S', 'priya@example.com', '9876543212', 'ACTIVE'),
('Karthik R', 'karthik@example.com', '9876543213', 'ACTIVE');

```

7.3 Insert Books

```

INSERT INTO books
(title, isbn, author_id, category, publisher, publication_year, total_copies, available_copies)
VALUES
('Clean Code', '9780132350884', 1, 'Programming', 'Prentice Hall', 2008, 5, 5),
('Atomic Habits', '9780735211292', 2, 'Self Help', 'Avery', 2018, 4, 4),
('Refactoring', '9780134757599', 3, 'Programming', 'Addison-Wesley', 2018, 3, 3),
('Effective Java', '9780134685991', 4, 'Programming', 'Addison-Wesley', 2018, 4, 4),
('The Pragmatic Programmer', '9780135957059', 5, 'Programming', 'Addison-Wesley', 2019, 2, 2);

```

7.4 Insert Borrowing Records

```
INSERT INTO borrowings
(user_id, book_id, issue_date, due_date, return_date, status)
VALUES
(1, 1, '2026-09-01', '2026-09-15', NULL, 'BORROWED'),
(2, 2, '2026-08-20', '2026-09-03', '2026-09-05', 'RETURNED'),
(3, 3, '2026-09-04', '2026-09-18', NULL, 'BORROWED');
```

8. CRUD Operations

CRUD means Create, Read, Update and Delete. These operations will be triggered by the web interface and executed through Flask API endpoints.

8.1 Create

```
-- Add a user
INSERT INTO users (name, email, phone)
VALUES ('Meena R', 'meena@example.com', '9876543220');

-- Add an author
INSERT INTO authors (author_name)
VALUES ('Brian Kernighan');

-- Add a book
INSERT INTO books
(title, isbn, author_id, category, publisher, publication_year, total_copies, available_copies)
VALUES
('The C Programming Language', '9780131103627', 6, 'Programming', 'Pearson', 1988, 3, 3);
```

8.2 Read

```
-- All books
SELECT * FROM books;

-- Books with author names
SELECT
    b.book_id, b.title, a.author_name, b.category,
    b.total_copies, b.available_copies
FROM books b
JOIN authors a ON b.author_id = a.author_id
ORDER BY b.title;

-- Search books by title
SELECT *
FROM books
WHERE title LIKE '%java%';
```

8.3 Update

```
-- Update user phone
UPDATE users
SET phone = '9999999999'
WHERE user_id = 1;

-- Update book details
UPDATE books
SET category = 'Programming', publisher = 'New Publisher'
WHERE book_id = 1;

-- Update user status
UPDATE users
SET status = 'INACTIVE'
WHERE user_id = 4;
```

8.4 Delete / Deactivate

For production-like behavior, permanent deletion should be avoided when history exists. Deactivation is safer for users and books. Permanent DELETE can still be used for test data that has no dependent records.

```
-- Safer book deactivation is implemented by adding a status field if desired.
-- For the current schema, delete only when no borrowing records reference the book.
DELETE FROM authors
WHERE author_id = 6;
```

9. Book Borrowing and Return Workflow

9.1 Borrow Book

The borrow operation must update two tables as one logical unit: insert the transaction and reduce the number of available copies. The application should use a transaction so that both operations succeed together or both are rolled back.

```
START TRANSACTION;

-- Check availability
SELECT available_copies
FROM books
WHERE book_id = 1
FOR UPDATE;

-- Create borrowing record
INSERT INTO borrowings
(user_id, book_id, issue_date, due_date, status)
VALUES
(1, 1, CURRENT_DATE, DATE_ADD(CURRENT_DATE, INTERVAL 14 DAY), 'BORROWED');

-- Reduce available copies
UPDATE books
SET available_copies = available_copies - 1
WHERE book_id = 1
AND available_copies > 0;

COMMIT;
```

9.2 Return Book

```
START TRANSACTION;

-- Mark borrowing as returned
UPDATE borrowings
SET return_date = CURRENT_DATE,
    status = 'RETURNED'
WHERE borrowing_id = 1
    AND status = 'BORROWED';

-- Increase available copies
UPDATE books
SET available_copies = available_copies + 1
WHERE book_id = (
    SELECT book_id FROM borrowings WHERE borrowing_id = 1
);

COMMIT;
```

9.3 Fine Calculation Without a Fines Table

The first version does not store fines in a separate table. A fine can be calculated when required from the number of days returned after the due date. Example policy: ₹5 per late day.

```
-- Fine for a returned book: ₹5 per late day
SELECT
    borrowing_id,
    GREATEST(DATEDIFF(return_date, due_date), 0) AS late_days,
    GREATEST(DATEDIFF(return_date, due_date), 0) * 5 AS fine_amount
FROM borrowings
WHERE return_date IS NOT NULL;
```

10. SQL Analysis and Reporting Queries

The reporting queries below are adapted to the normalized four-table schema. They cover common library questions such as most borrowed books, top borrowers, overdue books, and available inventory.

10.1 Total Books by Category

```
SELECT category, COUNT(*) AS total_titles
FROM books
GROUP BY category
ORDER BY total_titles DESC;
```

10.2 Top 5 Borrowers

```
SELECT
    u.user_id,
    u.name,
    COUNT(br.borrowing_id) AS books_borrowed
FROM users u
JOIN borrowings br ON u.user_id = br.user_id
GROUP BY u.user_id, u.name
ORDER BY books_borrowed DESC
LIMIT 5;
```

10.3 Most Borrowed Books

```
SELECT
    b.book_id,
    b.title,
    COUNT(br.borrowing_id) AS borrow_count
FROM books b
JOIN borrowings br ON b.book_id = br.book_id
GROUP BY b.book_id, b.title
ORDER BY borrow_count DESC
LIMIT 5;
```

10.4 Currently Overdue Books

```
SELECT
    br.borrowing_id,
    b.title,
    u.name AS borrower,
    br.due_date,
    DATEDIFF(CURRENT_DATE, br.due_date) AS overdue_days
FROM borrowings br
JOIN books b ON br.book_id = b.book_id
JOIN users u ON br.user_id = u.user_id
WHERE br.due_date < CURRENT_DATE
    AND br.return_date IS NULL
ORDER BY br.due_date ASC;
```

10.5 Late Returns

```
SELECT
    b.title,
    u.name AS borrower,
    br.due_date,
    br.return_date,
    DATEDIFF(br.return_date, br.due_date) AS late_days
FROM borrowings br
JOIN books b ON br.book_id = b.book_id
JOIN users u ON br.user_id = u.user_id
WHERE br.return_date > br.due_date
ORDER BY br.return_date DESC;
```

10.6 Available Inventory

```
SELECT
    title,
    total_copies,
    available_copies,
    (total_copies - available_copies) AS currently_issued
FROM books
ORDER BY title;
```

10.7 Dashboard KPIs

```
-- Total titles
SELECT COUNT(*) AS total_titles FROM books;

-- Total copies
SELECT COALESCE(SUM(total_copies), 0) AS total_copies FROM books;

-- Copies currently borrowed
SELECT COALESCE(SUM(total_copies - available_copies), 0) AS issued_copies
FROM books;

-- Active users
SELECT COUNT(*) AS active_users
FROM users
WHERE status = 'ACTIVE';

-- Active borrowing transactions
SELECT COUNT(*) AS active_borrowings
FROM borrowings
WHERE status = 'BORROWED';
```

11. Flask Integration

Flask should expose REST endpoints that map directly to the main CRUD and workflow operations. The frontend JavaScript can call these endpoints using fetch().

HTTP Method	Endpoint	Purpose
GET	/api/books	List all books
GET	/api/books/<id>	Get one book
POST	/api/books	Create a book
PUT	/api/books/<id>	Update a book
DELETE	/api/books/<id>	Delete/deactivate a book
GET	/api/users	List users
POST	/api/users	Create user
PUT	/api/users/<id>	Update user
DELETE	/api/users/<id>	Deactivate user
POST	/api/borrow	Issue a book
POST	/api/return/<id>	Return a book
GET	/api/borrowings	List borrowing history
GET	/api/borrowings/overdue	List overdue books
GET	/api/dashboard	Return dashboard KPIs

11.1 Suggested Flask Project Structure

```
library-management/
|-- app.py
|-- config.py
|-- requirements.txt
|-- schema.sql
|-- routes/
|   |-- books.py
|   |-- users.py
|   |-- borrowings.py
|   `-- dashboard.py
|-- templates/
|   `-- index.html
`-- static/
    |-- css/
    |   `-- style.css
    `-- js/
        `-- app.js
```

11.2 Database Connection

```
# Example environment variables
DB_HOST=your-rds-endpoint
DB_PORT=3306
DB_NAME=library_management
DB_USER=library_admin
DB_PASSWORD=your_strong_password
SECRET_KEY=your_flask_secret
```

Use environment variables or AWS Secrets Manager for credentials. Never commit database passwords into GitHub.

12. AWS Deployment Design

```
Internet
|
v
Nginx on EC2
|
v
Gunicorn
|
v
Flask Application
|
| private network / security group
v
AWS RDS MySQL
```

- EC2 hosts the Flask application, Gunicorn and Nginx.
- RDS hosts the MySQL database and keeps it separate from the application server.
- RDS should not be opened to the public internet; allow port 3306 only from the application server security group.
- Use HTTPS for the web application in production, typically through Nginx and a certificate.
- Store application secrets outside the source code.

13. Security and Data Integrity

- Use parameterized SQL queries or SQLAlchemy to prevent SQL injection.
- Hash passwords; never store plain-text passwords.
- Validate all request fields at the Flask API boundary.
- Use database transactions for borrow/return operations.
- Use foreign keys to protect relationships.
- Use least-privilege database credentials for the application.
- Back up the production database and monitor the RDS instance.

14. Testing Checklist

Area	Test Case	Expected Result
Books	Add a valid book	Book is inserted and receives an ID
Books	Duplicate ISBN	Insert is rejected
Users	Duplicate email	Insert is rejected
Borrow	Borrow when available_copies > 0	Transaction succeeds and available_copies decreases
Borrow	Borrow when available_copies = 0	Borrow is rejected
Return	Return active borrowing	return_date is set and copies increase
Return	Return same record twice	Second return is rejected/ignored
Overdue	Due date in past + return_date NULL	Record appears in overdue report
Integrity	Delete referenced author	Delete is blocked by FK
Search	Search title with partial text	Matching books are returned

15. Future Enhancements

- Role-based authentication for admin, librarian and member users.
- Separate category and publisher master tables when the project grows.
- Support multiple authors per book with a book_authors junction table.
- Add a fines/payments table when fine payment tracking is required.
- Email or SMS reminders for overdue books.
- Dashboard charts using Chart.js.
- Search, pagination and filtering for large catalogs.
- HTTPS, custom domain and CI/CD deployment on AWS.

16. Conclusion

The proposed four-table MySQL schema provides a practical foundation for a full-stack Library Management System. It is small enough for a student project but rich enough to demonstrate relational database concepts, CRUD operations, foreign keys, indexing, joins, transactions, reporting queries and backend integration. The schema can be used directly as the database layer for a Flask application and deployed with Flask on AWS EC2 and MySQL on AWS RDS.

17. Reference

GeeksforGeeks. “Library Management Data Analysis using SQL.” Last updated 15 Dec 2025. The reference article covers library book/borrower fields and SQL analysis of borrowing patterns, late returns, overdue books and fines.

<https://www.geeksforgeeks.org/sql/library-management-data-analysis-using-sql/>

End of Report